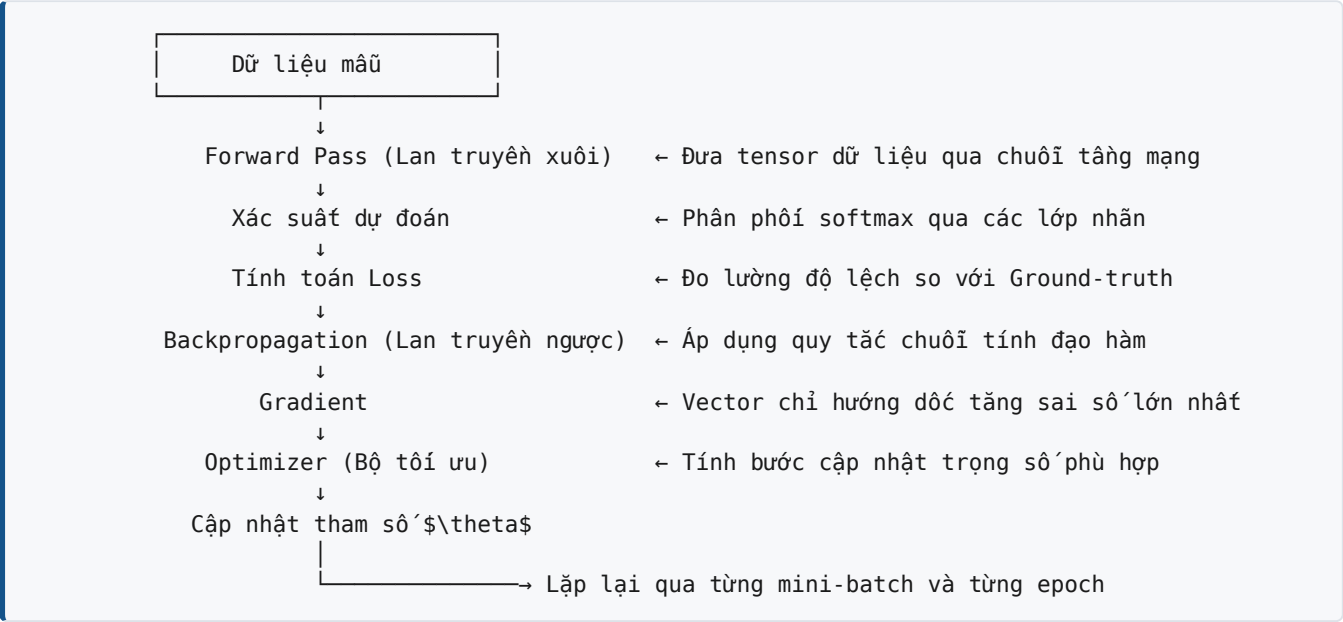


Trụ cột	Bản chất toán học / kỹ thuật	Biểu hiện trong đồ án
Kiến trúc mô hình	Xây dựng không gian hàm ánh xạ $f(x; \theta)$	MLP (bảng số), CNN & LeNet-5 (ảnh số)
Hàm mất mát (Loss)	Định lượng sai số giữa phân phối dự đoán và thực tế	Cross-Entropy Loss
Giải thuật tối ưu	Chiến lược dịch chuyển không gian trọng số θ	SGD có quán tính / Adam
Tập dữ liệu	Phân phối mẫu làm cơ sở học thống kê	Diabetes 130, MNIST, CIFAR-10

1.3 Quy trình học máy — Vòng tuần hoàn tối ưu



1.4 Trực quan hóa bản chất Gradient

Hình dung bạn là **một người bộ hành bị bịt mắt đang dò dẫm trên sườn núi phức tạp**, với mục đích tìm đường xuống đáy thung lũng sâu nhất (điểm cực tiểu của hàm mất mát) giữa sương mù bao phủ. Không thể quan sát bức tranh toàn cảnh, bạn chỉ có thể thăm dò qua độ dốc dưới chân: "*Nghiêng bàn chân về góc nào thì độ dốc đi xuống là gắt nhất?*"

Hướng đi dốc xuống nhanh nhất đó chính là **chiều ngược của Gradient**.

Khái niệm thực tế	Đại lượng trong mạng nơ-ron
Tọa độ người bộ hành	Tập tham số trọng số $\theta = (W, b)$
Độ cao vị trí hiện tại	Giá trị hàm mất mát (Loss L)
Chiều dốc đi xuống	Chiều ngược gradient — ∇L
Độ dài sải bước	Tốc độ học (Learning Rate η)

Quy tắc hiệu chỉnh tham số căn bản:

$$W_{\text{mới}} = W_{\text{cũ}} - \eta \times (\partial L / \partial W)$$

Vai trò của việc điều tiết Tốc độ học: - Nếu η quá nhỏ ($1e-6$): Quá trình hội tụ diễn ra chậm chạp, dễ kẹt ở các điểm yên ngựa. - Nếu η quá lớn (0.5): Bước nhảy vượt qua đáy thung lũng, dẫn đến hiện tượng dao động phân kỳ. - Nếu η phù hợp ($1e-3$ với Adam): Mạng tiếp cận vùng cực tiểu ổn định và trơn tru.

1.5 Backpropagation — Quy tắc chuỗi của phép vi phân

Một ngộ nhận thường gặp là đồng nhất lan truyền ngược với phép tính nguyên hàm. Thực tế, **Backpropagation là việc ứng dụng triệt để Quy tắc chuỗi (Chain Rule) của phép vi phân đa biến:**

$$x \xrightarrow{W_1} y \xrightarrow{W_2} z \longrightarrow L \text{ (Loss)}$$

$$\partial L / \partial W_1 = (\partial L / \partial z) \times (\partial z / \partial y) \times (\partial y / \partial W_1)$$

Tín hiệu sai số được truyền ngược từ ngõ ra về từng nút tham số ở ngõ vào.

Cần phân định rạch ròi: - **Backpropagation**: Chịu trách nhiệm **tính toán độ dốc** ($\partial L / \partial \theta$) cho từng tham số. - **Gradient Descent / Adam**: Chịu trách nhiệm **áp dụng** độ dốc đó để tịnh tiến tọa độ tham số.

Trong thư viện [scratch_nn.py](#), toàn bộ phép lan truyền ngược được hiện thực hóa tường minh bằng NumPy thông qua phương thức `backward()` và giải thuật `col2im`, minh chứng rõ ràng cho cơ chế toán học bên dưới lớp vỏ bọc autograd.

2. Giải phẫu Mạng Tích chập (CNN)

2.1 Vì sao ảnh số đòi hỏi CNN thay vì MLP

Một ảnh màu CIFAR-10 có độ phân giải khiêm tốn $32 \times 32 \times 3 = 3,072$ giá trị điểm ảnh. Nếu duỗi phẳng và nối thẳng vào tầng ẩn Dense đầu tiên có 1,024 nơ-ron:

$3,072 \times 1,024 \approx 3.15$ triệu trọng số – chỉ riêng một tầng đơn lẻ

Mạng kết nối dày đặc (Fully Connected) lập tức gặp ba rào cản chí mạng: 1. **Bùng nổ số lượng tham số**: Dẫn tới hiện tượng quá khớp (overfitting) nghiêm trọng và đòi hỏi tài nguyên bộ nhớ khổng lồ. 2. **Phá vỡ cấu trúc không gian cục bộ**: Phép duỗi phẳng (flatten) làm đứt gãy mối liên hệ lân cận giữa các điểm ảnh liền kề. 3. **Thiếu tính bất biến với phép dịch chuyển (Translation Invariance)**: Một chi tiết trôi lệch vài pixel bị nhận định như một đặc trưng hoàn toàn xa lạ.

CNN giải quyết triệt để hạn chế này nhờ **chia sẻ trọng số (Weight Sharing)** và **trường tiếp nhận cục bộ (Local Receptive Fields)**: một ma trận lọc kích thước nhỏ (như 3×3) được trượt tuần tự qua toàn bộ không gian ảnh.

Minh chứng thực nghiệm: Mạng `SmallCNN` trên CIFAR-10 chỉ cần **545,098 tham số**, còn biến thể `M1` tinh gọn chỉ sử dụng **72,730 tham số** nhưng đạt độ chính xác gần 70-72%, vượt trội hoàn toàn so với việc kết nối hàng triệu trọng số rời rạc.

2.2 Phép Tích chập (Convolution) — Trọng tâm trích xuất đặc trưng

Kernel là ma trận trọng số nhỏ học được ($K_H \times K_W$), trượt quét trên ảnh đầu vào, thực hiện nhân từng phần tử tương ứng rồi cộng dồn:

Vùng ảnh cục bộ	Kernel lọc
$\begin{bmatrix} 1 & 2 \\ 4 & 5 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$
* (nhân từng phần tử)	
$= (1 \times 1) + (2 \times 0) + (4 \times 0) + (5 \times (-1)) = -4$	

Kết quả của thao tác trượt quét không gian tạo ra một **Feature Map (Bản đồ đặc trưng)**. Điểm cốt lõi: **Các hệ số trong kernel không phải là hằng số định sẵn**. Chúng được mô hình tự học thông qua quá trình lan truyền ngược và cập nhật gradient.

2.3 Bước trượt (Stride) và Đệm viền (Padding)

- **Stride**: Khoảng cách dịch chuyển của cửa sổ trượt sau mỗi phép tính.
- **Stride = 1**: Quét chi tiết từng điểm ảnh, duy trì độ phân giải cao.
- **Stride = 2**: Bước nhảy cách quãng, giảm chiều không gian đi một nửa.
- **Padding**: Bổ sung một dải viền (thường bằng 0) bao bọc xung quanh ma trận.
- **valid** (không đệm): Kích thước ngõ ra giảm dần: $O = \lfloor (W - K) / S \rfloor + 1$.

- **same** (đệm phù hợp): Bảo toàn nguyên vẹn độ phân giải không gian gốc.
- Lợi ích: Ngăn ngừa suy hao kích thước quá nhanh và giúp các pixel vùng biên tham gia vào nhiều lượt trích xuất hơn.

2.4 Hàm phi tuyến ReLU

$$\text{ReLU}(x) = \max(0, x)$$

Nếu thiếu vắng hàm kích hoạt phi tuyến, việc xếp chồng nhiều tầng tích chập hay tuyến tính vẫn chỉ quy về **một phép biến đổi affine duy nhất** ($W_2(W_1x) = W_{combined}x$). ReLU cung cấp tính phi tuyến mạnh mẽ, triệt tiêu hiện tượng bão hòa gradient ở miền dương (giải quyết vanishing gradient), đồng thời sở hữu chi phí tính toán phần cứng cực kỳ tinh gọn.

2.5 Tầng lấy mẫu cực đại Max Pooling

Lớp Max Pooling 2×2 chọn lọc phản hồi mạnh nhất trong từng cửa sổ không gian:

Feature Map ngõ vào		Ngõ ra thu gọn
1324 5612 7293 4158	→	64 79

Ý nghĩa cấu trúc: 1. **Thu hẹp kích thước không gian**: Cắt giảm 75% số lượng điểm ảnh, tiết kiệm đáng kể bộ nhớ và số phép tính FLOPs. 2. **Thiết lập tính bất biến cục bộ**: Khi đặc trưng bị xê dịch nhẹ vài pixel, giá trị cực đại trong cửa sổ vẫn giữ nguyên.

2.6 Khối Flatten, Fully Connected và Softmax

- **Flatten**: Chuyển đổi tensor không gian đa chiều (C, H, W) thành vector 1 chiều $(C \cdot H \cdot W)$.
- **Fully Connected**: Hợp nhất và phối hợp các đặc trưng phân tán để hình thành biểu diễn trừu tượng bậc cao.
- **Softmax**: Chuẩn hóa các giá trị logits thô thành phân phối xác suất hợp lệ (tổng chuẩn hóa bằng 1.0):

$$\sigma(z)_i = \frac{e^{z_i}}{\sum e^{z_j}}$$

2.7 Bảng tổng hợp chức năng các tầng mạng

Thành phần	Cơ chế vận hành	Mục tiêu cốt lõi
Input	Tensor đa kênh (C, H, W)	Cung cấp tín hiệu dữ liệu chuẩn hoá
Conv2D	Tích chập cục bộ + chia sẻ trọng số	Trích chọn đặc trưng không gian
ReLU	Ngưỡng hoá $f(x) = \max(0, x)$	Cung cấp tính phi tuyến cho biểu diễn
MaxPool2D	Trích cực đại trong vùng 2×2	Giảm chiều, tạo tính bất biến dịch chuyển
Flatten	Tái định hình tensor thành vector	Cầu nối giữa khối trích xuất và phân loại
Dense	Phép nhân ma trận $xW + b$	Tổng hợp tương quan đặc trưng toàn cục
Softmax	Hàm mũ chuẩn hoá	Xuất ra phân phối xác suất phân lớp

2.8 Phân cấp trừu tượng của đặc trưng thị giác

Tầng thấp (C1)	→	Bắt các cạnh viền, góc nhọn, vệt sáng đơn giản
Tầng giữa (C2)	→	Hợp nhất thành hoa văn, đường cong, kết cấu bề mặt
Tầng cao (C3)	→	Nhận dạng các bộ phận hoàn chỉnh (bánh xe, tai động vật, cánh máy bay)
Tầng Dense	→	Khái quát thành thực thể ngữ nghĩa trọn vẹn (Ô tô, Con mèo, Tàu thủy)

3. Kỹ thuật Cải tiến Kiến trúc (Improved CNN Models)

3.1 Hạn chế của kiến trúc CNN thô sơ

Một cấu trúc CNN truyền thống cơ bản dạng `Conv → ReLU → Pool → Conv → ReLU → Pool → Dense` :

- Rất dễ rơi vào trạng thái **quá khớp (overfitting)** khi huấn luyện lâu. - **Trôi dạt phân phối kích hoạt nội tại (Internal Covariate Shift)** làm suy giảm tốc độ hội tụ. - Thiếu các đường dẫn tắt khiến tín hiệu gradient bị tiêu tán khi mở rộng độ sâu mạng (Degradation Problem). - Đánh giá vai trò của các kênh đặc trưng một cách đồng đều, thiếu cơ chế tái điều hướng thích nghi.

3.2 Các cơ chế nâng cấp điển hình

1. **Data Augmentation**: Biến đổi hình học ảnh huấn luyện (lật ngang, xoay góc, cắt cúp) nhằm tăng cường độ khái quát hóa.
2. **Batch Normalization**: Chuẩn hóa kích hoạt của từng mini-batch về kỳ vọng 0 và phương sai 1, giúp ổn định mặt phẳng tối ưu gradient.
3. **Dropout**: Tắt ngẫu nhiên một tỷ lệ kết nối nơ-ron trong pha huấn luyện, ngăn chặn sự đồng thích nghi sai lệch giữa các tham số.
4. **Residual Connections (ResNet)**: Lối tắt $Y = \mathcal{F}(X) + X$ tạo xa lộ cho dòng gradient truyền ngược thông suốt mà không suy giảm.
5. **Channel Attention (Squeeze-and-Excitation)**: Tự động điều chỉnh trọng số mức độ quan trọng giữa các kênh đặc trưng.

3.3 Phân tích thực nghiệm: Bốn biến thể trên CIFAR-10 (Full 50,000)

Thực nghiệm được tiến hành đồng nhất trên môi trường PyTorch (RTX 3060 GPU), 10 epochs, seed cố định, cấu trúc kênh hẹp `[16, 32, 64]` kết hợp Global Average Pooling:

Biến thể	Cơ chế tích hợp	Tham số	Train Loss	Test Accuracy	Macro-F1	Thời gian (s)	Độ chênh lệch vs M1
M1	Conv + ReLU + Pool (Gốc)	72,730	0.8074	69.55%	0.7001	19.5	—
M2	+ Batch Normalization	72,954	0.4579	74.86%	0.7483	22.1	+5.31%
M3	+ Residual Shortcut	75,786	0.4479	75.74%	0.7634	27.6	+6.19%
M4	+ SE Attention	78,614	0.4335	75.25%	0.7557	38.3	+5.70%

Những phát hiện then chốt từ dữ liệu thực nghiệm:

1. **Batch Normalization mang lại bước nhảy vọt lớn nhất**: Chỉ tiêu tốn thêm vốn vẹn **224 tham số** (+0.3%), nhưng **M₂** nâng độ chính xác tăng vọt **+5.31 điểm phần trăm** (từ 69.55%

lên 74.86%). Train loss giảm từ 0.8074 xuống 0.4579, khẳng định sự ổn định hóa phân phối kích hoạt thúc đẩy tốc độ hội tụ cực kỳ hiệu quả.

2. Residual Connection mang lại cải tiến ổn định: M_3 xác lập mức hiệu năng cao nhất bảng (75.74%, tăng thêm 0.88% so với M_2). Ở độ sâu khiêm tốn gồm 3 khối chức năng, hiện tượng tiêu tán gradient chưa xuất hiện trầm trọng, do đó giá trị cốt lõi của residual ở đây là cung cấp thêm luồng thông tin đa tuyến hơn là giải cứu sự suy thoái gradient.

3. Hiện tượng quá khớp rõ nét tại biến thể Attention M4: Điểm quan trọng nhất trong chuỗi thực nghiệm nằm ở việc M_4 đạt Train Loss thấp nhất toàn bảng (0.4335) nhưng Test Accuracy lại giảm sút xuống 75.25% (thua kém M_3 khoảng 0.49 điểm). Đây là minh chứng kinh điển của **Overfitting**: - Khối SE bổ sung thêm các tầng Fully Connected thu phóng kênh, tăng tính phi tuyến và độ phức tạp cục bộ. - Trong giới hạn 10 epochs và số kênh tương đối hẹp (16/32/64), cơ chế attention chưa tích lũy đủ dữ liệu tối ưu để học được mặt nạ kênh ổn định, dẫn đến việc mô hình khớp vào các nhiễu thống kê của tập train.

Đúc kết kiến trúc: Sự phức tạp hóa mô hình không đồng nghĩa với khả năng tổng quát hóa. Các cơ chế tiên tiến chỉ mang lại giá trị thực chất khi mô hình thực sự chịu đựng điểm nghẽn tương ứng; nếu không, chúng sẽ trở thành chi phí dư thừa về tham số và thời gian tính toán.

3.4 Sức mạnh của quy mô dữ liệu

So sánh năng lực mô hình `SmallCNN` khi thay đổi quy mô dữ liệu huấn luyện:

Quy mô dữ liệu	PyTorch Test Acc	Keras Test Acc
CIFAR-10 (Subset 5,000 ảnh)	52.90%	54.50%
CIFAR-10 (Toàn phần 50,000 ảnh)	71.84%	70.11%

Gia tăng dữ liệu gấp 10 lần giúp độ chính xác bật tăng **khoảng 16-19 điểm phần trăm**, trong khi cấu trúc kiến trúc không thay đổi một dòng lệnh nào. Điều này khẳng định chân lý: *"Dữ liệu phong phú là điều kiện tiên quyết trước khi tính đến các cải tiến tinh vi về vi kiến trúc."*

4. Đúc kết và Đọc dữ liệu Phản biện

4.1 Ba tiêu điểm định lượng cốt lõi

- 1. **Tính tương đồng thuật toán:** Trên tập con MNIST 10k, Scratch NumPy đạt 98.10% còn PyTorch đạt 98.35% — độ lệch chỉ 0.25% nằm hoàn toàn trong phạm vi dao động của hạt giống ngẫu nhiên ban đầu.
- 2. **Hiệu năng cơ chế cải tiến:** Trên CIFAR-10, bước nhảy từ M_1 (69.55%) lên M_3 (75.74%) đem lại mức tăng **+6.19%** nhờ kết hợp chuẩn hoá batch và lỗi tắt tắt dòng thông tin.
- 3. **Ưu thế gia tốc phần cứng:** Trên các phép toán tích chập đa chiều, PyTorch khai thác GPU RTX 3060 mang lại tốc độ thực thi vượt trội gấp hàng chục lần so với vòng lặp tuần tự của NumPy trên CPU.

4.2 Cạm bẫy chỉ số đánh giá trên tập dữ liệu Diabetes 130

Khung làm việc	Accuracy	Macro-F1
Scratch (NumPy)	60.79%	0.3462
TensorFlow/Keras	60.77%	0.3686
PyTorch	61.05%	0.3694

Chỉ số Accuracy đạt ~61% thoát nhìn có vẻ ổn thỏa, song **Macro-F1 chỉ dao động trong khoảng ~0.35-0.37**. Hiện tượng này phản ánh sự **lệch pha nghiêm trọng do mất cân bằng lớp (Class Imbalance)**: mô hình có xu hướng thiên vị dự đoán vào nhóm chiếm đại đa số (bệnh nhân không tái nhập viện) và bỏ qua các ca tái nhập viện sớm (<30 ngày). Nếu chỉ dựa vào Accuracy, chúng ta sẽ ngộ nhận hoàn toàn về giá trị ứng dụng thực tiễn của mô hình.

5. Giải đáp các Câu hỏi Cốt lõi

Q: Vì sao ReLU được ưu tiên vượt trội so với Sigmoid trong các tầng ẩn? *Trả lời:*

Sigmoid có đạo hàm đạt cực đại chỉ 0.25 và tiệm cận dần về 0 ở hai biên, khiến tích các gradient qua chuỗi nhiều tầng suy giảm nhanh chóng về 0 (Vanishing Gradient). ReLU duy trì hệ số góc bằng 1 không đổi ở miền dương, giúp tín hiệu gradient truyền ngược thông suốt mà không suy hao.

Q: Thao tác Pooling có làm thất thoát thông tin không? *Trả lời:* Có làm mất thông tin tọa độ điểm ảnh chính xác, song đổi lại việc loại bỏ chi tiết cục bộ giúp cắt giảm khối lượng tính toán và rèn luyện cho mạng tính bất biến nhận dạng khi đối tượng bị dịch chuyển nhẹ.

Q: Khác biệt bản chất giữa Backpropagation và Gradient Descent là gì? *Trả lời:*

Backpropagation đảm trách **tính toán** giá trị độ dốc gradient của hàm mất mát đối với từng trọng số thông qua quy tắc chuỗi vi phân. Gradient Descent đảm trách **hiệu chỉnh** vị trí trọng số dựa trên các giá trị gradient đã tính.

Q: Tại sao mô hình M4 có cấu trúc tinh vi hơn nhưng kết quả kiểm thử lại kém hơn M3? *Trả lời:* Do hiện tượng quá khớp (overfitting). Khối SE bổ sung thêm các tầng thu phóng kênh làm tăng dung lượng tham số và tính phi tuyến, nhưng 10 epochs chưa đủ dài để mạng tối ưu hóa các trọng số chú ý, dẫn đến việc mô hình bị phân tâm bởi nhiễu tập train và giảm năng lực khái quát hóa trên tập test.

Phụ lục: Quy trình Tái lập Kết quả

```
# Huấn luyện và lưu bộ nhớ đệm các biến thể kiến trúc M1..M4
python prefill_variants.py

# Dựng lại báo cáo tổng hợp README.md
python make_report.py --no-pdf

# Kết xuất toàn bộ các tài liệu lý thuyết và báo cáo chuyên đề sang định dạng PDF
python make_docs_pdf.py
```